

SQL Week 0 — Querying

Student Learning Guide

Source: Harvard CS50 SQL — cs50.harvard.edu/sql/2024/notes/0

Database: `longlist.db` — International Booker Prize Longlists (2018–2023)

What You Will Learn

By the end of this guide you will be able to:

- Ask a database for exactly the data you need using `SELECT`
- Filter results using `WHERE` , `LIKE` , `BETWEEN` , and `NULL` checks
- Sort and limit results using `ORDER BY` and `LIMIT`
- Calculate stats using aggregate functions (`AVG` , `COUNT` , `MAX` , `MIN` , `SUM`)
- Rename output columns using `AS`

No prior SQL knowledge needed. You only need your curiosity.

The Database We Are Using

CS50 uses a real dataset: **books longlisted for the International Booker Prize** from 2018 to 2023.

The `longlist` table has these columns:

Column	What it contains
<code>title</code>	Book title
<code>author</code>	Author name
<code>translator</code>	Translator name (can be empty)
<code>year</code>	Year longlisted
<code>format</code>	hardcover, paperback, etc.
<code>rating</code>	Reader rating (e.g. 3.85)
<code>votes</code>	Number of ratings
<code>pages</code>	Number of pages
<code>publisher</code>	Publisher name

Tool: DB Browser for SQLite

To run queries, use one of these tools:

Tool	How to use
DB Browser for SQLite	Download from sqlitebrowser.org → Open longlist.db → "Execute SQL" tab
SQLiteOnline.com	Open in browser → Upload longlist.db → type queries
CS50.dev	Official cloud environment — no install needed

Before writing a single query — open the database and look at the data.

Run `SELECT * FROM "longlist";` and spend 2 minutes reading the rows.

Concept 1 — SELECT

What question does it answer?

"What data is in this table?"

How it works

SELECT tells the database **which columns** you want.

FROM tells it **which table** to look in.

```
-- All columns from the longlist table
SELECT * FROM "longlist";

-- Only book titles
SELECT "title" FROM "longlist";

-- Titles and their authors
SELECT "title", "author" FROM "longlist";

-- Titles, authors, and translators
SELECT "title", "author", "translator" FROM "longlist";
```

Rule: ***** means "all columns." Always use double quotes around column and table names in SQLite.

Concept 1 — Try It Yourself

Open `longlist.db` and run each query. After each one, ask yourself:

1. Run `SELECT * FROM "longlist";`
→ How many columns do you see? What are they?
2. Run `SELECT "title" FROM "longlist";`
→ How is this different from the previous result?
3. Run `SELECT "title", "author" FROM "longlist";`
→ Which two columns appear?
4. **Challenge:** Write a query to show only the `rating` and `votes` columns.

Concept 2 — LIMIT

What question does it answer?

"Can I just see the first few rows?"

How it works

`LIMIT` restricts how many rows are returned. Useful when a table has thousands of rows.

```
-- Show only the first 3 books
SELECT "title", "author" FROM "longlist" LIMIT 3;

-- Show only the first 10 books
SELECT "title", "author" FROM "longlist" LIMIT 10;
```

Why it matters: Real databases have millions of rows.

Without `LIMIT`, every query could return 1,000,000 results.

Concept 2 — Try It Yourself

1. Run `SELECT "title" FROM "longlist" LIMIT 5;`

→ Which 5 books appear?

2. Run `SELECT "title", "author" FROM "longlist" LIMIT 1;`

→ What is the first book in the database?

3. **Think about it:** What determines which rows come first?

(Hint: databases don't guarantee order unless you ask for it — we'll fix this with `ORDER BY` soon.)

Concept 3 — WHERE

What question does it answer?

"Show me only the rows that match a condition."

How it works

`WHERE` filters rows before returning them. Only rows where the condition is `true` are included.

```
-- Books longlisted in 2023
SELECT "title", "author" FROM "longlist" WHERE "year" = 2023;

-- All books by a specific author
SELECT "title", "author" FROM "longlist" WHERE "author" = 'Fernanda Melchor';

-- Books NOT released in hardcover
SELECT "title", "format" FROM "longlist" WHERE "format" != 'hardcover';

-- Same result, different syntax
SELECT "title", "format" FROM "longlist" WHERE "format" <> 'hardcover';

-- Same result, using NOT
SELECT "title", "format" FROM "longlist" WHERE NOT "format" = 'hardcover';
```

WHERE — Comparison Operators

Operator	Meaning	Example
<code>=</code>	Equal to	<code>"year" = 2023</code>
<code>!=</code> or <code><></code>	Not equal to	<code>"format" != 'hardcover'</code>
<code><</code>	Less than	<code>"pages" < 300</code>
<code>></code>	Greater than	<code>"rating" > 4.0</code>
<code><=</code>	Less than or equal	<code>"year" <= 2022</code>
<code>>=</code>	Greater than or equal	<code>"votes" >= 10000</code>

Note: Text values use single quotes `'like this'`.
Column names use double quotes `"like this"`.

Concept 3 — Try It Yourself

1. Find all books longlisted in 2022.
2. Find all books with more than 4.0 rating.
3. Find all books that are NOT in paperback format.
4. Find all books with fewer than 300 pages.
5. **Challenge:** Find all books with a rating above 4.0 AND more than 10,000 votes.

```
-- Template
SELECT "title", "rating", "votes" FROM "longlist"
WHERE "rating" > 4.0 AND "votes" > 10000;
```

Concept 4 — AND, OR, NOT (Compound Conditions)

What question does it answer?

"What if I need more than one condition?"

```
-- Books longlisted in 2022 OR 2023
SELECT "title", "year" FROM "longlist"
WHERE "year" = 2022 OR "year" = 2023;

-- Books longlisted in 2022 or 2023 AND not in hardcover
SELECT "title", "year", "format" FROM "longlist"
WHERE ("year" = 2022 OR "year" = 2023) AND "format" != 'hardcover';
```

Important: Use parentheses when mixing `AND` and `OR`.

Without them, SQL follows operator precedence and may give unexpected results.

Concept 4 — Try It Yourself

1. Find books from 2019 or 2020.
2. Find books from 2021 that are in hardcover format.
3. Find books with a rating above 4.2 OR more than 50,000 votes.
4. **Think about it:** What is the difference between these two queries?

```
-- Query A
SELECT "title" FROM "longlist"
WHERE "year" = 2022 OR "year" = 2023 AND "format" = 'hardcover';

-- Query B
SELECT "title" FROM "longlist"
WHERE ("year" = 2022 OR "year" = 2023) AND "format" = 'hardcover';
```

Run both. Do they return the same results? Why or why not?

Concept 5 — NULL

What question does it answer?

"How do I find rows where a value is missing?"

How it works

`NULL` means **unknown** or **no value**. Not zero. Not an empty string.

You cannot use `=` to check for NULL — you must use `IS NULL` or `IS NOT NULL`.

```
-- Books that were NOT translated (no translator)
SELECT "title", "translator" FROM "longlist"
WHERE "translator" IS NULL;

-- Books that WERE translated (have a translator)
SELECT "title", "translator" FROM "longlist"
WHERE "translator" IS NOT NULL;
```

Concept 5 — NULL ⚠ Common Mistake

This is the single most common error beginners make with NULL:

❌ Wrong	✅ Correct
WHERE "translator" = NULL	WHERE "translator" IS NULL
WHERE "translator" != NULL	WHERE "translator" IS NOT NULL

Why does `= NULL` not work?

Because NULL means *unknown*. SQL cannot confirm that *unknown* equals *unknown* — so the comparison always returns nothing. You need `IS NULL` which is a special check, not a comparison.

Think of it this way: if someone asks "Is your missing wallet equal to my missing wallet?" — you can't answer yes or no. You can only say "both are missing."

Concept 5 — Try It Yourself

1. Run `SELECT "title", "translator" FROM "longlist" WHERE "translator" IS NULL;`
→ How many books have no translator?
2. Run `SELECT "title", "translator" FROM "longlist" WHERE "translator" IS NOT NULL;`
→ How many books were translated?
3. **Think about it:** Why might a book have no translator?
What does it tell you about that book?
4. **Common mistake test:** Try running `WHERE "translator" = NULL .`
What happens? Does it return any results? Why?

Concept 6 — LIKE

What question does it answer?

"How do I search for text that partially matches?"

How it works

LIKE lets you match patterns using two wildcards:

- % — matches zero or more characters (anything)
- _ — matches exactly one character

```
-- Books with "love" anywhere in the title
SELECT "title" FROM "longlist" WHERE "title" LIKE '%love%';

-- Books whose title starts with "The" (including "There", "They")
SELECT "title" FROM "longlist" WHERE "title" LIKE 'The%';

-- Books whose title starts with the WORD "The"
SELECT "title" FROM "longlist" WHERE "title" LIKE 'The %';

-- Titles matching "P_re" – like "Pyre" or "Pire" (unsure of spelling)
SELECT "title" FROM "longlist" WHERE "title" LIKE 'P_re';
```

LIKE — The Difference Between % and _

Pattern	Matches	Does NOT match
'The%'	"The", "There", "The Almond"	"A The"
'The %'	"The Almond", "The Hours"	"There", "Theatre"
'%love%'	"Lovely War", "I love SQL", "love"	"Ive"
'P_re'	"Pyre", "Pare", "Pure"	"Perre", "Pre"
'%love'	"Deeply in love", "love"	"love story"

Concept 6 — Try It Yourself

1. Find all books with "the" in their title (case insensitive in SQLite).
2. Find all books starting with the word "A".
3. Find all books ending with "s".
4. Find all books where the author's last name starts with "M".

```
-- Template: author's last name starts with M
SELECT "title", "author" FROM "longlist"
WHERE "author" LIKE '% M%';
```

5. **Challenge:** Find books whose title starts with "The" AND contains "love".

```
SELECT "title" FROM "longlist"
WHERE "title" LIKE 'The%love%';
```

Concept 7 — BETWEEN + Range Conditions

What question does it answer?

"How do I filter a range of values?"

```
-- Verbose way: books from 2019 to 2022
SELECT "title", "year" FROM "longlist"
WHERE "year" >= 2019 AND "year" <= 2022;

-- Cleaner way using BETWEEN (inclusive on both ends)
SELECT "title", "year" FROM "longlist"
WHERE "year" BETWEEN 2019 AND 2022;

-- Books with high rating
SELECT "title", "rating" FROM "longlist"
WHERE "rating" > 4.0;

-- Popular AND highly rated books
SELECT "title", "rating", "votes" FROM "longlist"
WHERE "rating" > 4.0 AND "votes" > 10000;

-- Short books
SELECT "title", "pages" FROM "longlist"
WHERE "pages" < 300;
```

Concept 7 — Try It Yourself

1. Find books longlisted between 2020 and 2022.
2. Find books with a rating between 3.5 and 4.0.
3. Find books with more than 500 pages.
4. Find books longlisted in 2023 with fewer than 250 pages.
5. **Think about it:** Is `BETWEEN 2019 AND 2022` the same as `>= 2019 AND <= 2022` ?
Test it — do both return the same number of rows?

Concept 8 — ORDER BY

What question does it answer?

"How do I sort my results?"

```
-- Sort by rating, lowest first (ascending – default)
SELECT "title", "rating" FROM "longlist"
ORDER BY "rating" LIMIT 10;

-- Sort by rating, highest first (descending)
SELECT "title", "rating" FROM "longlist"
ORDER BY "rating" DESC LIMIT 10;

-- Sort by rating DESC, break ties by votes DESC
SELECT "title", "rating", "votes" FROM "longlist"
ORDER BY "rating" DESC, "votes" DESC LIMIT 10;

-- Sort alphabetically by title
SELECT "title" FROM "longlist"
ORDER BY "title";

-- Highly-rated books, sorted – combining WHERE + ORDER BY + LIMIT
SELECT "title", "rating" FROM "longlist"
WHERE "votes" > 10000
ORDER BY "rating" DESC LIMIT 10;
```

ORDER BY — Key Rules

Rule	Example
Default order is <code>ASC</code> (lowest to highest)	<code>ORDER BY "rating"</code>
Use <code>DESC</code> for highest to lowest	<code>ORDER BY "rating" DESC</code>
Sort by multiple columns — second column breaks ties	<code>ORDER BY "rating" DESC, "votes" DESC</code>
<code>ORDER BY</code> goes AFTER <code>WHERE</code>	<code>WHERE ... ORDER BY ...</code>
<code>LIMIT</code> goes AFTER <code>ORDER BY</code>	<code>ORDER BY ... LIMIT 10</code>

Important pattern: `WHERE` → `ORDER BY` → `LIMIT`

This order is fixed. Never swap them.

Concept 8 — Try It Yourself

1. Find the 5 books with the highest rating.
2. Find the 5 books with the most votes.
3. Find the 10 shortest books (fewest pages), sorted shortest first.
4. Find the 3 most recent books (latest year).
5. **Challenge:** Find the top 5 most-voted books longlisted after 2020.

```
-- Template
SELECT "title", "votes", "year" FROM "longlist"
WHERE "year" > 2020
ORDER BY "votes" DESC
LIMIT 5;
```

Concept 9 — Aggregate Functions

What question does it answer?

"How do I calculate stats across all rows?"

Aggregate functions collapse many rows into a single result.

```
-- Average rating of all books
SELECT AVG("rating") FROM "longlist";

-- Rounded to 2 decimal places
SELECT ROUND(AVG("rating"), 2) FROM "longlist";

-- Highest rating
SELECT MAX("rating") FROM "longlist";

-- Lowest rating
SELECT MIN("rating") FROM "longlist";

-- Total votes across all books
SELECT SUM("votes") FROM "longlist";

-- Total number of books
SELECT COUNT(*) FROM "longlist";
```

COUNT — Three Important Variations

```
-- Count all rows (includes NULLs)
SELECT COUNT(*) FROM "longlist";

-- Count only rows where translator is NOT NULL
SELECT COUNT("translator") FROM "longlist";

-- Count unique publishers (no duplicates)
SELECT COUNT(DISTINCT "publisher") FROM "longlist";
```

Key difference:

- `COUNT(*)` → counts every row
- `COUNT("column")` → counts only non-NULL values in that column
- `COUNT(DISTINCT "column")` → counts unique values only

Run all three and compare the numbers. Why are they different?

Concept 10 — AS

What question does it answer?

"How do I give my result column a meaningful name?"

Without `AS`, aggregate results get an ugly auto-generated column name. `AS` renames it.

```
-- Without AS - ugly column name
SELECT ROUND(AVG("rating"), 2) FROM "longlist";

-- With AS - clean column name
SELECT ROUND(AVG("rating"), 2) AS "average rating" FROM "longlist";

-- More examples
SELECT MAX("rating") AS "highest rating" FROM "longlist";
SELECT MIN("rating") AS "lowest rating" FROM "longlist";
SELECT COUNT(*) AS "total books" FROM "longlist";
SELECT COUNT(DISTINCT "publisher") AS "unique publishers" FROM "longlist";
```

Why it matters: When your SQL feeds into a web app, the column name becomes the key in your JSON response. Clean names = clean APIs.

Concept 9 + 10 — Try It Yourself

1. What is the average number of pages across all books?
2. What is the highest number of votes any single book received?
3. How many books have a translator listed?
4. How many unique authors are in the longlist?
5. Show the average rating, rounded to 2 decimal places, with the label "avg rating".

```
SELECT ROUND(AVG("rating"), 2) AS "avg rating" FROM "longlist";
```

6. **Challenge:** What is the average rating of books that have more than 10,000 votes?

Putting It All Together

The full query structure — in this exact order:

```
SELECT  "columns"           -- What columns do you want?
FROM    "table"             -- Which table?
WHERE   "condition"         -- Which rows? (filter first)
ORDER BY "column" DESC      -- In what order?
LIMIT  10;                  -- How many rows?
```

Example — Top 10 translated books by rating with at least 1000 votes:

```
SELECT
    "title",
    "author",
    "translator",
    ROUND("rating", 2) AS "rating"
FROM "longlist"
WHERE "translator" IS NOT NULL
    AND "votes" > 1000
ORDER BY "rating" DESC
LIMIT 10;
```

Practice Set — Mix of All Concepts

Work through these in order. Each one builds on the previous.

Level 1 — Basic

1. Show all book titles and their year.
2. Show the 10 most recent books.
3. Find all books from 2021.
4. Find books with more than 400 pages.

Level 2 — Filtering

5. Find books published in hardcover with a rating above 4.0.
6. Find books that were NOT translated (no translator).
7. Find books whose title contains the word "night".
8. Find books between 2019 and 2021 with fewer than 300 pages.

Practice Set Continued

Level 3 — Sorting + Aggregates

9. Show the top 5 highest-rated books that have been translated.
10. How many books were longlisted in total?
11. What is the average rating of all hardcover books?
12. How many unique authors appear across all years?

Level 4 — Challenge

13. Show the title and rating of the single lowest-rated book.
14. Which year had the most books longlisted? (*Hint: this needs GROUP BY — coming in Session 2*)
15. Find all books by an author whose last name you only partially remember — it starts with "Holm".

Common Mistakes — Know These Before They Trip You Up

Mistake	Wrong	Correct
NULL comparison	<code>WHERE "translator" = NULL</code>	<code>WHERE "translator" IS NULL</code>
Wrong quote type	<code>WHERE title = "python"</code>	<code>WHERE "title" = 'python'</code>
Missing wildcard	<code>WHERE "title" LIKE 'love'</code>	<code>WHERE "title" LIKE '%love%'</code>
Wrong clause order	<code>LIMIT 10 WHERE "year" = 2023</code>	<code>WHERE "year" = 2023 LIMIT 10</code>
<code>COUNT(col)</code> includes NULL	Assuming it counts all rows	<code>COUNT(*)</code> for all, <code>COUNT(col)</code> for non-NULL
Mixing AND/OR without brackets	Unexpected results	Always use <code>()</code> when combining AND + OR
<code>=</code> for text patterns	<code>WHERE "title" = 'The%'</code>	<code>WHERE "title" LIKE 'The%'</code>

Quick Reference Card

```
-- Basic retrieval
SELECT "col1", "col2" FROM "table";
SELECT * FROM "table";
SELECT "col" FROM "table" LIMIT 10;

-- Filtering
WHERE "col" = 'value'
WHERE "col" != 'value'
WHERE "col" > 4.0 AND "col2" < 300
WHERE "col" BETWEEN 2019 AND 2022
WHERE "col" LIKE '%pattern%'
WHERE "col" IS NULL
WHERE "col" IS NOT NULL

-- Sorting
ORDER BY "col" ASC      -- low to high (default)
ORDER BY "col" DESC     -- high to low

-- Aggregates
COUNT(*)              -- all rows
COUNT("col")          -- non-NULL values only
COUNT(DISTINCT "col") -- unique values only
AVG("col")
MAX("col") / MIN("col")
SUM("col")
ROUND(AVG("col"), 2) AS "label"
```

What's Next — Session 2: Relating

You've learned to query **one table**.

But real applications have **many tables** that connect to each other:

- Users table + Posts table
- Students table + Courses table
- Customers table + Orders table

Session 2 teaches you how to link tables, combine data, and answer questions that span multiple tables — using `JOIN`, subqueries, and keys.

Preparation question for next class:

Think of an app you use daily — Instagram, WhatsApp, or your university system. How many separate tables do you think its database has? What are they?

Session Resources

Resource	Link
CS50 SQL Week 0 Notes	cs50.harvard.edu/sql/2024/notes/0
CS50 Week 0 Source Code	cs50.harvard.edu/sql/2024/weeks/0 → Source Code
longlist.db download	Available in Source Code zip
DB Browser for SQLite	sqlitebrowser.org
SQLiteOnline (no install)	sqliteonline.com
CS50 Cloud Environment	cs50.dev